Compiler Design

CHAPTER 6

Intermediate Code Generation

Neso Academy

Compiler Design

Intermediate Codes

Outcome

- ☆ Understanding Intermediate Codes.
- ☆ Classification of Intermediate Codes.

Compiler – Internal Architecture

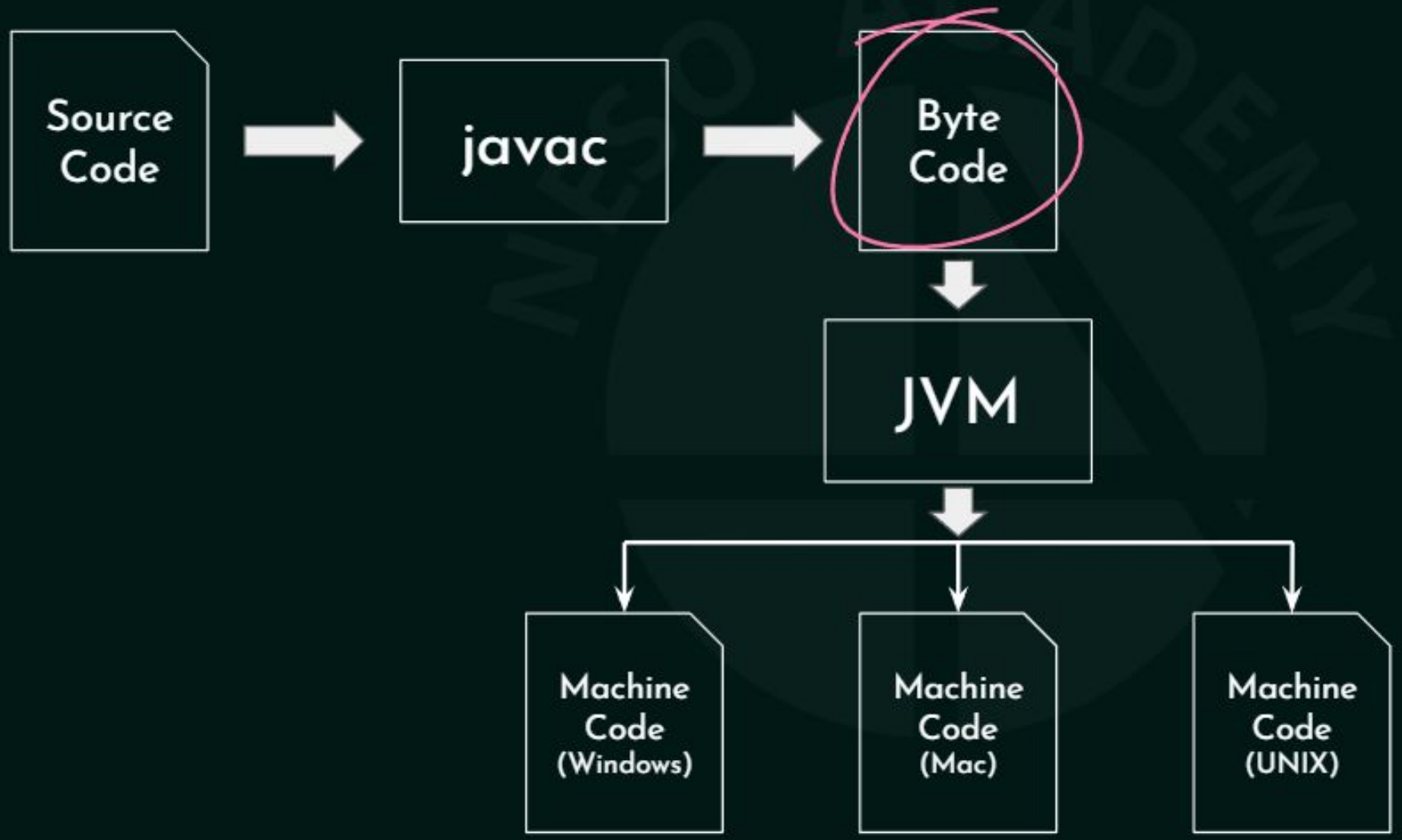
- Lexical Analysis
- Syntax Analysis
- Semantic Analysis
- Intermediate Code Generation

} Front-End

- Code Optimization
- Target Code Generation

} Back-End

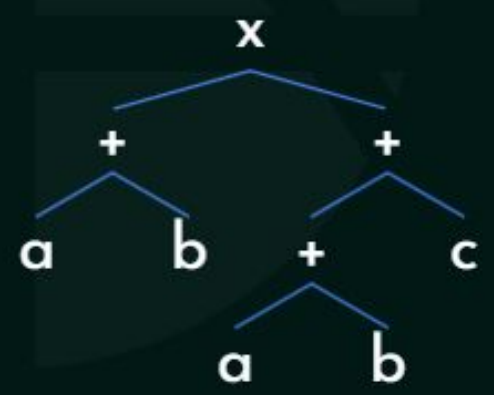
How Java works:



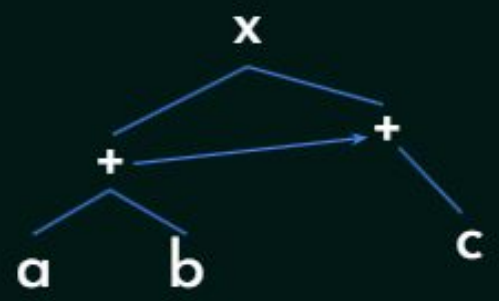
Classification of Intermediate Codes:

Expression: (a+b)x(a+b+c)

- Intermediate Codes
  - Linear Form
    - Postfix Expression
      - ab+ab+c+x
    - Three Address Code (TAC)
      - $t_1 = a + b$
      - $t_2 = a + b$
      - $t_3 = t_2 + c$
      - $t_4 = t_1 \times t_3$
  - Tree Form
    - Syntax Tree



    - Directed Acyclic Graph (DAG)



Compiler Design

Three Address Codes (TAC)

Outcome

- ☆ Different forms of TAC.
- ☆ Various representations of TAC.

Different forms of TAC:

The HLL expressions are converted into the intermediate code using any of the following TAC forms.

TAC Form	Usage
1. x = y op z	For Binary Operation & then assignment.
2. x = op z	For Unary Operation & then assignment.
3. x = y	For simple assignment.
4. if (x <rel op> y) GOTO L	Conditional GOTO.
5. GOTO L	Unconditional GOTO.
6. A[i] = x y = A[i]	Used for arrays.
7. x = *p y = &x	'x' is a value pointed by the pointer 'p'. Address of 'x' is stored in 'y'.

Representations of TAC:

Traditionally, three types of representations of TAC are popularly used.

1. Quadruples:
   These use temporary variables (in temporary registers) for each individual portion of the expression in order to store the result.
   Each instruction is splitted into the following 4 different fields:
   opr, op1, op2, result
2. Triples:
   For these temporary variables are not dedicated rather temporary registers are used on demand. The flow of evaluation is numbered and cannot be altered.
3. Indirect Triples:
   It uses pointers to the listing of all references to computations which is made separately and stored in memory.
   Temporaries are implicit and its rearrangeable like quadruples.

Representations of TAC:

Expression: -(a+b) x (c+d) + (a+b+c)

Three Address Code:

1. $t_1 = a + b$
2. $t_2 = -t_1$
3. $t_3 = c + d$
4. $t_4 = t_2 \times t_3$
5. $t_5 = a + b$
6. $t_6 = t_5 + c$
7. $t_7 = t_4 + t_6$

	Quadruples					Triples				Indirect Triples
	opr	op1	op2	result		opr	op1	op2		Reference
1.	+	a	b	t_1	1.	+	a	b	i.	(1)
2.	-	t_1		t_2	2.	-	(1)		ii.	(2)
3.	+	c	d	t_3	3.	+	c	d	iii.	(3)
4.	x	t_2	t_3	t_4	4.	x	(2)	(3)	iv.	(4)
5.	+	a	b	t_5	5.	+	a	b	v.	(5)
6.	+	t_5	c	t_6	6.	+	(5)	c	vi.	(6)
7.	+	t_4	t_6	t_7	7.	+	(4)	(6)	vii.	(7)

Representations of TAC:

Expression: -(a+b) x (c+d) + (a+b+c)

Three Address Code:

1. $t_1 = a + b$
2. $t_2 = -t_1$
3. $t_3 = c + d$
4. $t_4 = t_2 \times t_3$
5. $t_5 = a + b$
6. $t_6 = t_5 + c$
7. $t_7 = t_4 + t_6$

	Quadruples			
	opr	op1	op2	result
1.	+	a	b	t_1
2.	-	t_1		t_2
3.	+	c	d	t_3
4.	x	t_2	t_3	t_4
5.	+	a	b	t_5
6.	+	t_5	c	t_6
7.	+	t_4	t_6	t_7

Pro:

Statements can be rearranged.

Con:

Too many registers are needed.

Representations of TAC:

Three Address Code:

1. $t_1 = a + b$
2. $t_2 = -t_1$
3. $t_3 = c + d$
4. $t_4 = t_2 \times t_3$
5. $t_5 = a + b$
6. $t_6 = t_5 + c$
7. $t_7 = t_4 + t_6$

Pro:
Temporary registers are used on demand.

Expression: -(a+b) x (c+d) + (a+b+c)

	Triples		
	opr	op1	op2
1.	+	a	b
2.	-	(1)	b
3.	+	c	d
4.	x	(2)	(3)
5.	+	a	b
6.	+	(5)	c
7.	+	(4)	(6)

Con:
Statements can't be rearranged.

Representations of TAC:

Expression: -(a+b) x (c+d) + (a+b+c)

Three Address Code:

1. $t_1 = a + b$
2. $t_2 = -t_1$
3. $t_3 = c + d$
4. $t_4 = t_2 \times t_3$
5. $t_5 = a + b$
6. $t_6 = t_5 + c$
7. $t_7 = t_4 + t_6$

Pro: Statements comprise only references(i.e. pointers) ,thus, rearrangeable .

Con: Two memory accesses.

Indirect Triples	
	Reference
i.	(1)
ii.	(2)
iii.	(3)
iv.	(4)
v.	(5)
vi.	(6)
vii.	(7)

Compiler Design

Backpatching and Conversion to TAC

Outcome

- ☆ How backpatching works during the generation of TAC.
- ☆ Example of conversion from HLL to TAC.

Conversion to TAC:

```
if(a<b) then t = 1
else t = 0
```

4 Addresses

TAC Form
1. x = y op z
2. x = op z
3. x = y
4. if (x <rel op> y) GOTO L
5. GOTO L
6. A[i] = x y = A[i]
7. x = *p y = &x

Conversion to TAC:

```
if(a<b) then t = 1
else t = 0
```

```
i+0: if(a<b) GOTO (i+3)
i+1: t = 0
i+2: GOTO (i+4)
i+3: t = 1
i+4:
```

The procedure of leaving the labels empty and filling those later is called Back patching.

TAC Form
1. x = y op z
2. x = op z
3. x = y
4. if (x <rel op> y) GOTO L
5. GOTO L
6. A[i] = x y = A[i]
7. x = *p y = &x

Conversion to TAC – Example:

Convert the HLL: (a<b)&&(c<d)||(e<f) into TAC using Back patching. (Assume that absolute addressing is used from location 100 onwards).

t_1 will determine a<b
t_2 will determine c<d
t_3 will determine e<f
t_4 will determine t_1&t_2
t_5 will determine t_4|t_3

Conversion to TAC – Example:

Convert the HLL: (a<b)&&(c<d)||(e<f) into TAC using Back patching. (Assume that absolute addressing is used from location 100 onwards).

100: if(a<b) GOTO 103
101: $t_1 = 0$
102: GOTO 104
103: $t_1 = 1$
104: if(c<d) GOTO 107
105: $t_2 = 0$
106: GOTO 108
107: $t_2 = 1$
108: if(e<f) GOTO 111
109: $t_3 = 0$
110: GOTO 112
111: $t_3 = 1$
112: $t_4 = t_1 \& t_2$
113: $t_5 = t_4 | t_3$

Compiler Design

TAC – While Loop

Outcome

- ☆ Conversion of While Loop to TAC.
- ☆ Example Problem.

Different forms of TAC:

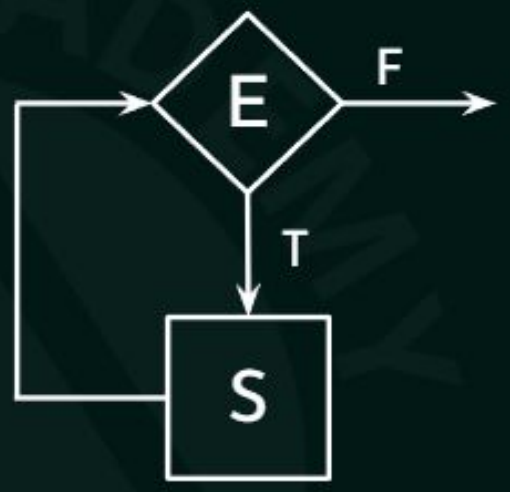
The HLL expressions are converted into the intermediate code using any of the following TAC forms.

TAC Form	Usage
1. x = y op z	For Binary Operation & then assignment.
2. x = op z	For Unary Operation & then assignment.
3. x = y	For simple assignment.
4. if (x <rel op> y) GOTO L	Conditional GOTO.
5. GOTO L	Unconditional GOTO.
6. A[i] = x y = A[i]	Used for arrays.
7. x = *p y = &x	'x' is a value pointed by the pointer 'p'. Address of 'x' is stored in 'y'.

While Loop:

- while(E) do S

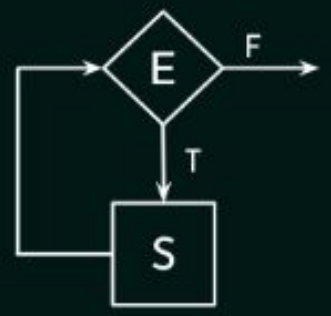
- ```
  while(E)
  {
      S
  }
  ```


  ```

While Loop:

Way 1:

```
L :  if(E==0) GOTO L1
     S
     GOTO L
L1 :
```

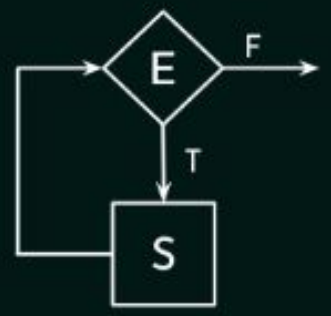


TAC Form
1. x = y op z
2. x = op z
3. x = y
4. if (x <rel op> y) GOTO L
5. GOTO L
6. A[i] = x y = A[i]
7. x = *p y = &x

While Loop:

Way 2:

```
L  : if(E) GOTO L1
     GOTO L2
L1 : S
     GOTO L
L2 :
```



TAC Form
1. x = y op z
2. x = op z
3. x = y
4. if (x <rel op> y) GOTO L
5. GOTO L
6. A[i] = x y = A[i]
7. x = *p y = &x

While Loop:

Way 1:

```
L:  if(E==0) GOTO L1
    S
    GOTO L
L1:
```

Way 2:

```
L:  if(E) GOTO L1
    GOTO L2
L1: S
    GOTO L
L2:
```

Note: After the HLL Loop has been converted to TAC, it is impossible to recognize it as it's been implemented using conditional and unconditional GOTO TAC statements.

Conversion to TAC – Example:

Convert the HLL to TAC:

```
while(x<y)
{
    a = b + c;
    x++;
}
```

```
L:  if(x<y) GOTO L1
    GOTO L2
L1: t1 = b + c
    a = t1
    t2 = x + 1
    x = t2
    GOTO L
L2:
```

Compiler Design

TAC – For Loop

Outcome

- ☆ Conversion of For Loop to TAC.

Different forms of TAC:

The HLL expressions are converted into the intermediate code using any of the following TAC forms.

TAC Form	Usage
1. x = y op z	For Binary Operation & then assignment.
2. x = op z	For Unary Operation & then assignment.
3. x = y	For simple assignment.
4. if (x <rel op> y) GOTO L	Conditional GOTO.
5. GOTO L	Unconditional GOTO.
6. A[i] = x y = A[i]	Used for arrays.
7. x = *p y = &x	'x' is a value pointed by the pointer 'p'. Address of 'x' is stored in 'y'.

While Loop:

Way 1:

```
L:  if(E==0) GOTO L1
    S
    GOTO L
L1:
```

Way 2:

```
L:  if(E) GOTO L1
    GOTO L2
L1: S
    GOTO L
L2:
```

Note: After the HLL Loop has been converted to TAC, it is impossible to recognize it as it's been implemented using conditional and unconditional GOTO TAC statements.

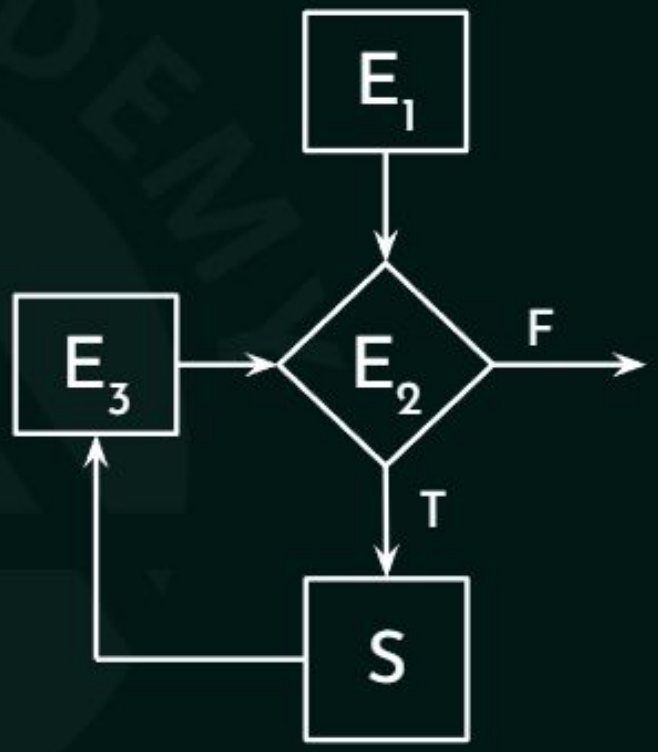
For Loop:

```
for(E1;E2;E3)
{
    S
}
```

E_1 : Initialization

E_2 : Condition

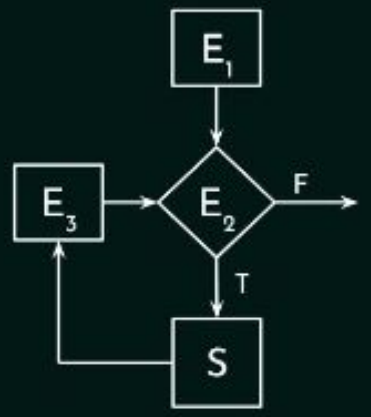
E_3 : Increment / Decrement



For Loop:

```
for(i=0;i<10;i++)
{
    a = b + c;
}
```

```
      i = 0
L:    if(i<10) GOTO L1
      GOTO L2
L1:   t1 = b + c
      a = t1
      t2 = i + 1
      i = t2
      GOTO L
L2:
```



TAC Form	
1.	x = y op z
2.	x = op z
3.	x = y
4.	if (x <rel op> y) GOTO L
5.	GOTO L
6.	A[i] = x y = A[i]
7.	x = *p y = &x

Compiler Design

TAC – Multiway Branching

Outcome

- ☆ Conversion of Switch-Case to TAC.

Conversion to TAC:

```
if(a<b) then t = 1
else t = 0
```

```
i+0: if(a<b) GOTO (i+3)
i+1: t = 0
i+2: GOTO (i+4)
i+3: t = 1
i+4:
```

The procedure of leaving the labels empty and filling those later is called Back patching.

TAC Form
1. x = y op z
2. x = op z
3. x = y
4. if (x <rel op> y) GOTO L
5. GOTO L
6. A[i] = x y = A[i]
7. x = *p y = &x

Conversion to TAC:

```
if(a<b) then t = 1
else t = 0
```

```
i+0: if(a<b) GOTO (i+3)
i+1: t = 0
i+2: GOTO (i+4)
i+3: t = 1
i+4:
```

The procedure of leaving the labels empty and filling those later is called Back patching.

TAC of Switch-Case:

```
switch(m+n)
{
    case(1): a=b+c;
             break;
    case(2): p=q+r;
             break;
    Default: x=y+z;
             break;
}
```

```
     t = m + n
     GOTO T
L1:  t1 = b + c
     a = t1
     GOTO L4
L2:  t2 = q + r
     p = t2
     GOTO L4
L3:  t3 = y + z
     x = t3
     GOTO L4
```

```
T: if(t==1) GOTO L1
   if(t==2) GOTO L2
   GOTO L3
L4:
```

Compiler Design

TAC – 2D Array

Outcome

- ☆ Understanding memory representation of 2D Arrays.
- ☆ Conversion of HLL code which accesses a 2D Array to TAC.

Different forms of TAC:

The HLL expressions are converted into the intermediate code using any of the following TAC forms.

TAC Form	Usage
1. x = y op z	For Binary Operation & then assignment.
2. x = op z	For Unary Operation & then assignment.
3. x = y	For simple assignment.
4. if (x <rel op> y) GOTO L	Conditional GOTO.
5. GOTO L	Unconditional GOTO.
6. A[i] = x y = A[i]	Used for arrays.
7. x = *p y = &x	'x' is a value pointed by the pointer 'p'. Address of 'x' is stored in 'y'.

2D Arrays:

A[3X3]:

00	01	02
10	11	12
20	21	22

RMO:

00	01	02	10	11	12	20	21	22

CMO:

00	10	20	01	11	21	02	12	22

2D Arrays:

A[3X3]:

00	01	02
10	11	12
20	21	22

RMO:

00	01	02	10	11	12	20	21	22
0	1	2	3	4	5	6	7	8

$$2 \times 3 + 1 = 7$$

2D Arrays:

A[10][20]

X = A[y,z]

Size of each cell = 4 units

$t_1 = y \times 20$

$t_2 = t_1 + z$

$t_3 = t_2 \times 4$

$t_4 =$ base Address of A

$X = t_4[t_3]$

A:

Compiler Design

TAC – Solved Problems (Set 1)

Outcome

- ☆ Two solved problems on TAC.

Q1: The least number of temporary variables required to create a three-address code in static single assignment form for the expression a = b x d - c + b x e - c is _____

GATE 1999

a. 3
b. 4
c. 5
d. 6

Q1: The least number of temporary variables required to create a three-address code in static single assignment form for the expression a = b x d - c + b x e - c is _____

a. 3
b. 4
c. 5
d. 6

$a = t_4$

$t_1 = b \times d$
$t_2 = b \times e$
$t_3 = t_1 + t_2$
$t_4 = t_3 - 2c$

Q2: In a simplified computer the instructions are:

OP R_j, R_i	Performs R_j OP R_i and stores the result in register R_j.
OP m, R_i	Performs Val OP R_i and stores the result in register R_i. Val is the content of memory location m.
MOV m, R_i	Moves the content of memory location m to register R_i.
MOV R_i, m	Moves the content of register R_i to memory location m.

The computer has only 2 registers and OP is either ADD or SUB. Consider the following basic block:

$t_1 = a + b$
$t_2 = c + d$
$t_3 = e - t_2$
$t_4 = t_1 - t_3$

GATE 2007

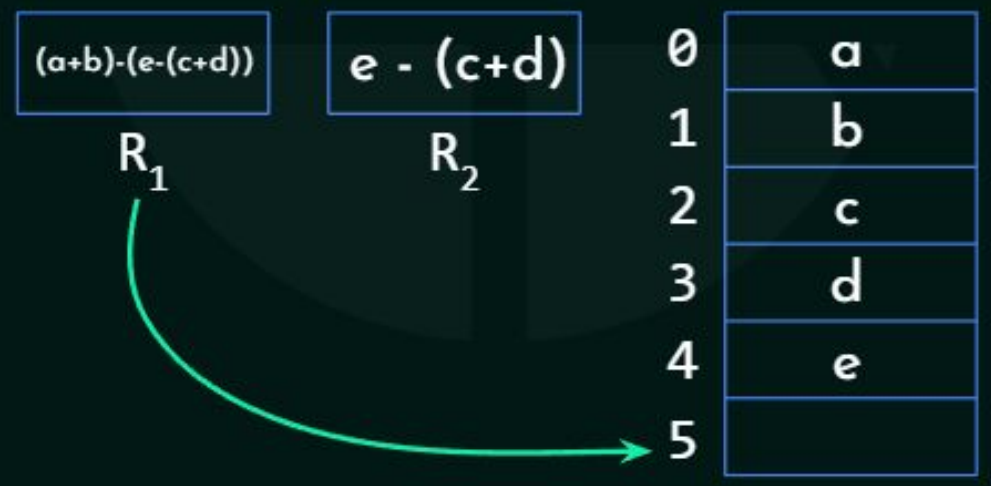
Assume that all operands are initially in memory. The final value of the computation should be in memory. What is the minimum number of MOV instructions in the code generated for this basic block?

Q2: In a simplified computer the instructions are:

OP R_j, R_i	Performs R_j OP R_i and stores the result in register R_j.
OP m, R_i	Performs Val OP R_i and stores the result in register R_i. Val is the content of memory location m.
MOV m, R_i	Moves the content of memory location m to register R_i.
MOV R_i, m	Moves the content of register R_i to memory location m.

The computer has only 2 registers and OP is either ADD or SUB.
Consider the following basic block:

$t_1 = a + b$
$t_2 = c + d$
$t_3 = e - t_2$
$t_4 = t_1 - t_3$




```
MOV 0, R1
MOV 2, R2
ADD 1, R1
ADD 3, R2
SUB 4, R2
SUB R1, R2
MOV R1, 5
```

Q2: In a simplified computer the instructions are:

OP R_j, R_i	Performs R_j OP R_i and stores the result in register R_j.
OP m, R_i	Performs Val OP R_i and stores the result in register R_i. Val is the content of memory location m.
MOV m, R_i	Moves the content of memory location m to register R_i.
MOV R_i, m	Moves the content of register R_i to memory location m.

The computer has only 2 registers and OP is either ADD or SUB.

Consider the following basic block:

$t_1 = a + b$
$t_2 = c + d$
$t_3 = e - t_2$
$t_4 = t_1 - t_3$

```
MOV 0, R1
MOV 2, R2
ADD 1, R1
ADD 3, R2
SUB 4, R2
SUB R1, R2
MOV R1, 5
```

Assume that all operands are initially in memory. The final value of the computation should be in memory. What is the minimum number of MOV instructions in the code generated for this basic block? Ans. 3

Compiler Design

TAC – Solved Problems (Set 2)

Outcome

- ☆ Two solved problems on TAC.

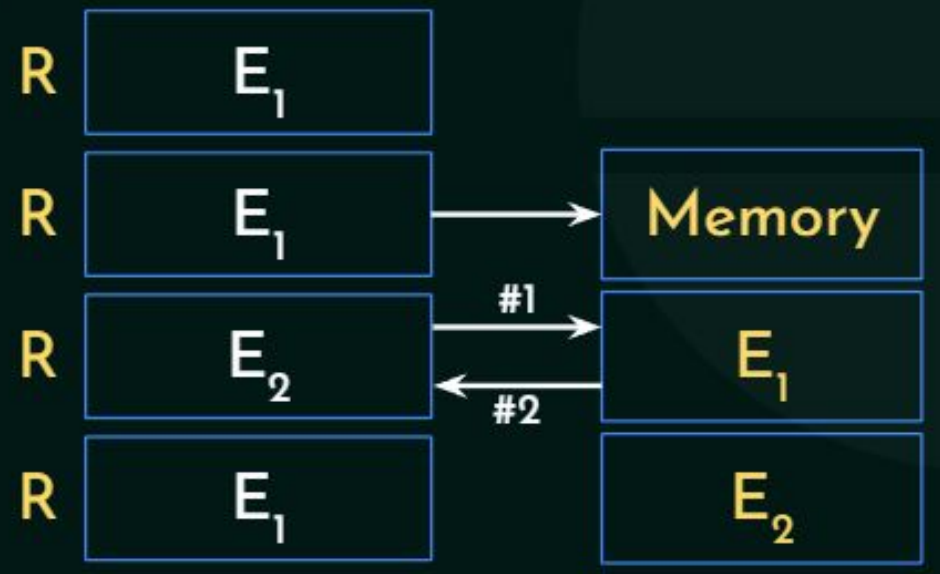
Q1: Consider the grammar rule $E \rightarrow E_1 - E_2$ for arithmetic expressions. The code generated is targeted to a CPU having a single user register. The subtraction operation requires the first operand to be in the register. If E_1 and E_2 do not have any common sub expression, in order to get the shortest possible code:

GATE 2004

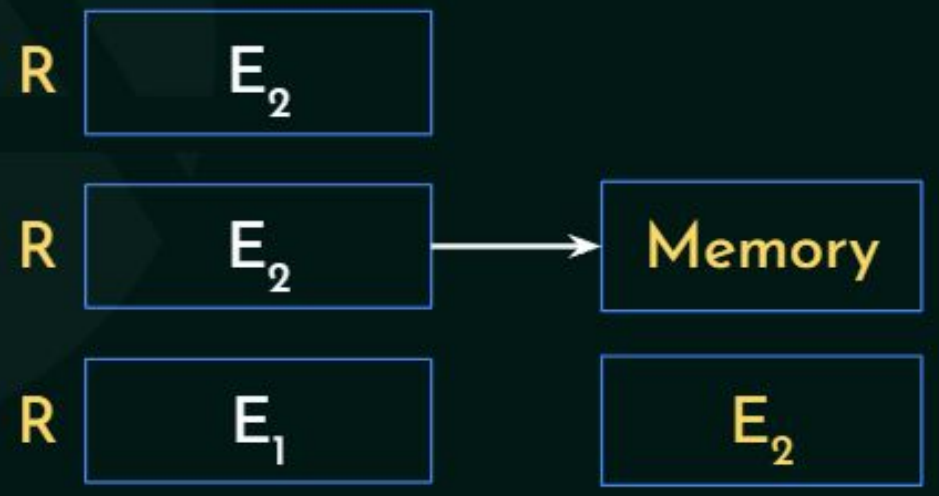
a. E_1 should be evaluated first.
b. E_2 should be evaluated first.
c. Evaluation of E_1 and E_2 should necessarily be interleaved.
d. Order of evaluation of E_1 and E_2 is of no consequence.

Q1: Consider the grammar rule $E \rightarrow E_1 - E_2$ for arithmetic expressions. The code generated is targeted to a CPU having a single user register. The subtraction operation requires the first operand to be in the register. If E_1 and E_2 do not have any common sub expression, in order to get the shortest possible code:

Sol. E_1 is evaluated first:



E_2 is evaluated first:

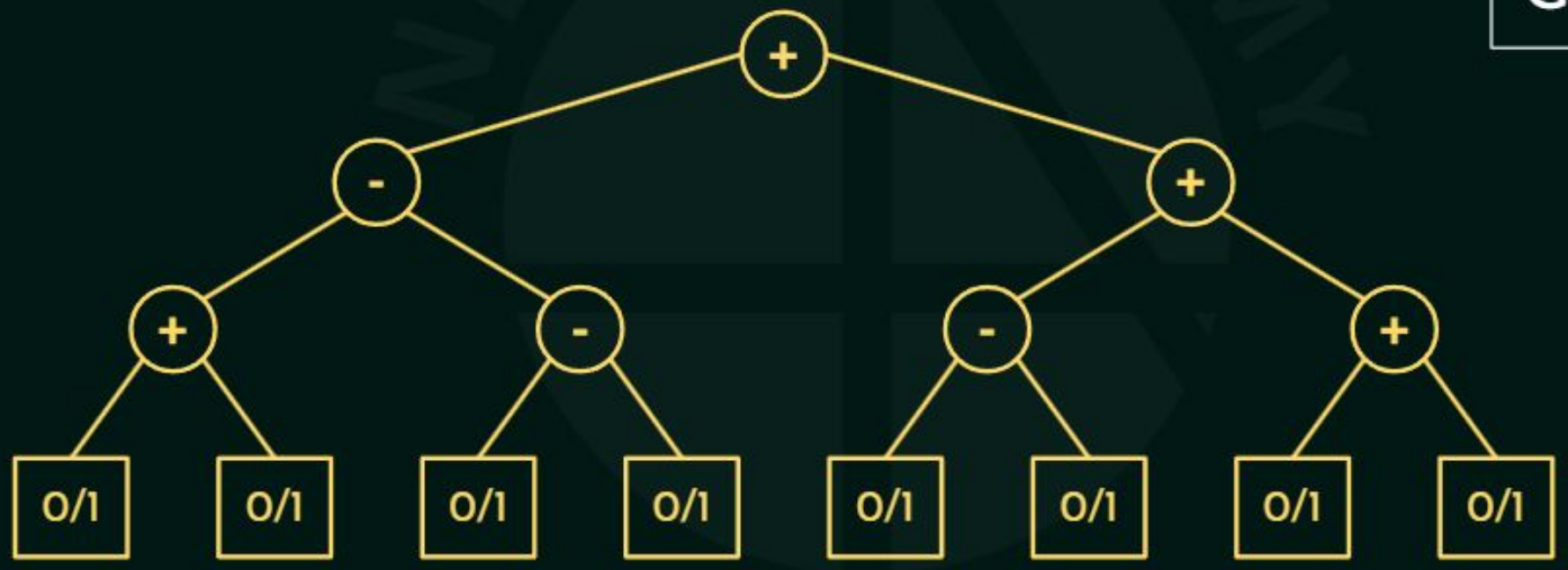


Q1: Consider the grammar rule $E \rightarrow E_1 - E_2$ for arithmetic expressions. The code generated is targeted to a CPU having a single user register. The subtraction operation requires the first operand to be in the register. If E_1 and E_2 do not have any common sub expression, in order to get the shortest possible code:

a. E_1 should be evaluated first.
b. E_2 should be evaluated first.
c. Evaluation of E_1 and E_2 should necessarily be interleaved.
d. Order of evaluation of E_1 and E_2 is of no consequence.

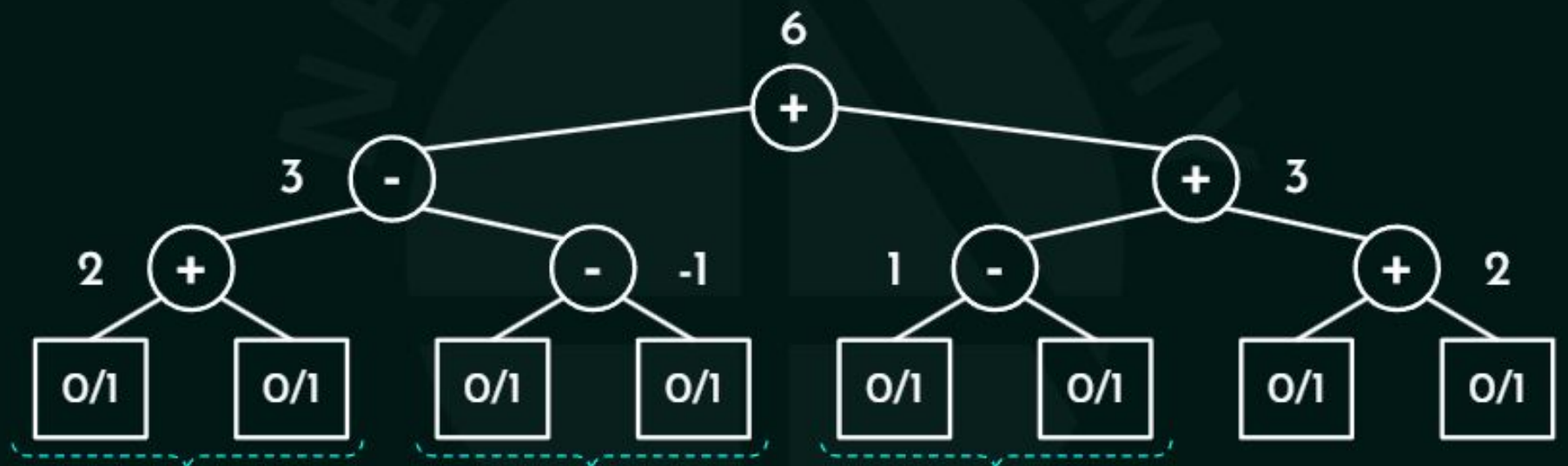
Q2: Consider the expression tree shown. Each leaf represents a numerical value, which can either be 0 or 1. Over all possible choices of the values at the leaves, the maximum possible value of the expression represented by the tree is _____

GATE 2014

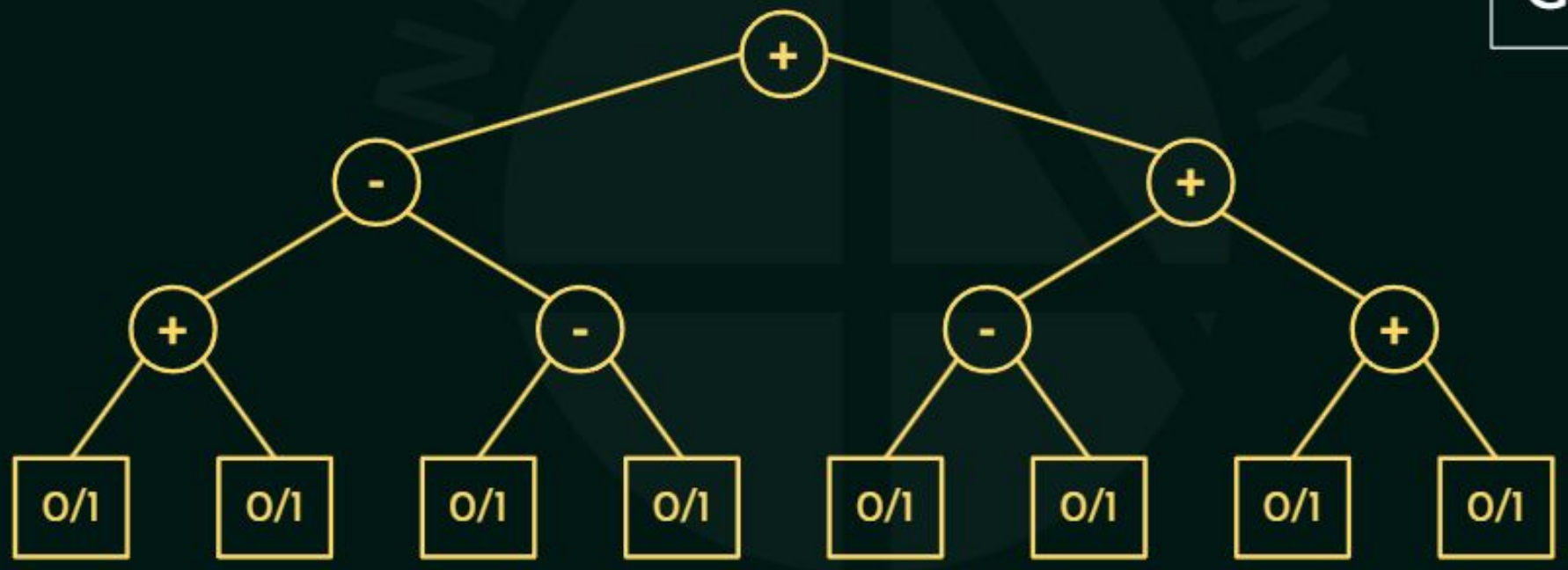


Q2: Consider the expression tree shown. Each leaf represents a numerical value, which can either be 0 or 1. Over all possible choices of the values at the leaves, the maximum possible value of the expression represented by the tree is _____

Sol.



0 + 0 = 0	0 - 0 = 0	0 - 0 = 0
0 + 1 = 1	0 - 1 = -1	0 - 1 = -1
1 + 0 = 1	1 - 0 = 1	1 - 0 = 1
1 + 1 = 2	1 - 1 = 0	1 - 1 = 0

Q2: Consider the expression tree shown. Each leaf represents a numerical value, which can either be 0 or 1. Over all possible choices of the values at the leaves, the maximum possible value of the expression represented by the tree is __6__

GATE 2014